

Smoothed Slices MATLAB App Specification

Version 2.0 - Comparative MATLAB demonstrator for Slices, NF-iSAM, and Smoothed Slices

AI-ready implementation brief: frontend, backend, UI/UX, diagnostics, figure generation, working-directory hierarchy, and incremental-process visualization

Non-negotiable purpose of the app

The app is not only a visualization of the proposed Smoothed Slices idea. It is a comparative case-study runner for three methods: (1) Slices Perspective, (2) NF-iSAM, and (3) Smoothed Slices. The core scientific purpose is to demonstrate how well each method recovers the full posterior distribution in classical non-Gaussian SLAM examples, and to explain why their internal inference processes differ.

1. What this updated file changes

- **Comparative purpose.** The app must run all three methods side by side, with the same datasets, same noise settings, same random seeds when possible, and matched support/sample budgets when scientifically meaningful.
- **Full LaTeX rendering policy.** All mathematical UI content must be created through components that support LaTeX interpretation. Axes titles, axes labels, tick labels, legends, equation labels, tooltips, and explanatory math cards must use LaTeX where MATLAB supports it.
- **Figure generator.** Add a dedicated button and tab for exporting every visible figure, every paper-style diagnostic figure, and a machine-readable diagnostics bundle for the current run.
- **Working-directory hierarchy.** The app must validate the project folder and explicitly know that implementation instruction documents live in the Instructions folder, while complementary papers live in Literature.
- **Process sliders.** Add timeline sliders for incremental step, elimination/training step, and method-specific process stage: variable elimination for Slices, Bayes-tree flow training/sampling for NF-iSAM, and conditional-smoothing-surface recursion for Smoothed Slices.
- **Paper-style figures.** Generate figures analogous to the Slices and NF-iSAM papers: factor graph to Bayes net/Bayes tree, marginal posterior evolution, posterior samples/KDE, runtime, RMSE, MMD, and sample/flow/surface diagnostics.

2. Directory hierarchy and source-of-truth contract

Project root. The user works inside a project directory with the following intended hierarchy. The AI implementing the app must not assume files are scattered arbitrarily. It must scan this hierarchy at startup and show a green/red validation card in the app.

```
Project/
|-- Instructions/
|   |-- nf_isam_exact_implementation.pdf
|   |-- slices_perspective_exact_implementation.pdf
|   |-- smoothed_slices_exact_implementation.pdf
|   |-- smoothed_slices_matlab_app_spec.pdf or .docx
|
|-- Literature/
|   |-- A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces*.pdf
|   |-- Incremental_Non-Gaussian_Inference_for_SLAM_Using_Normalizing_Flows*.pdf
|   |-- Batch Continuous-Time Trajectory Estimation*.pdf
|   |-- Probabilistic data association*.pdf
```

```

| |-- Other complementary papers used only for understanding, not as implementation authority
|
|-- presentation/
| |-- figures exported from the app
| |-- animations exported from the app
| |-- screenshots intended for slides
|
|-- data/           % create if missing
|-- results/        % create if missing
|-- src/            % MATLAB source code
|-- +methods/       % Slices, NF-iSAM, Smoothed Slices backends
|-- +viz/           % plotting and figure-export utilities
|-- +metrics/       % RMSE, MMD, ESS, runtime, cardinality diagnostics
|-- +datasets/      % synthetic and loaded SLAM case generators
|-- +utils/         % config, LaTeX styling, logging, reproducibility

```

Folder	Role in implementation	App behavior
Instructions	Authoritative method instructions written for the implementer.	Load and display document status. Provide clickable open buttons. Use this folder as the implementation reference panel.
Literature	Complementary papers used to understand schemes, figures, algorithms, and experimental conventions.	Load bibliographic list, but do not treat every paper as executable specification.
presentation	Output area for generated figures, animations, and slide-ready screenshots.	Figure Generator must save there by default, inside a run-specific subfolder.
data	Datasets, generated maps, synthetic scenarios, cached preprocessed measurements.	Create if missing. Allow user to select or regenerate case studies.
results	Run outputs, MAT files, JSON configs, metrics CSVs, diagnostics PDF/PNG.	Create run folder results/run_YYYYMMDD_HHMMSS_methodCompare.
src	App and backend MATLAB code.	All algorithms live outside the .mlapp when possible, to keep app callbacks thin.

Startup validation requirement

At startup, the app calls `validateProjectTree(projectRoot)`. The returned status structure must list missing required instruction files, missing literature files, missing source folders, and missing output folders. The UI must show a Project Health panel before any method is run.

```

function status = validateProjectTree(projectRoot)
    status.root = projectRoot;
    status.instructionsDir = fullfile(projectRoot,'Instructions');
    status.literatureDir = fullfile(projectRoot,'Literature');
    status.presentationDir = fullfile(projectRoot,'presentation');
    status.resultsDir = fullfile(projectRoot,'results');
    status.srcDir = fullfile(projectRoot,'src');
    status.requiredInstructions = {
        'slices_perspective_exact_implementation.pdf',
        'nf_isam_exact_implementation.pdf',
        'smoothed_slices_exact_implementation.pdf',
        'smoothed_slices_matlab_app_spec.pdf'};
    status = checkFilesAndCreateOutputFolders(status);
end

```

3. Scientific purpose: compare posterior recovery, not only trajectories

The app must be designed around distribution recovery. The three methods should be compared by their ability to recover the posterior shape, ambiguity, mode collapse, and uncertainty evolution. Trajectory error alone is insufficient because the point mean or MAP can hide posterior-shape errors.

Method	What the app must demonstrate	Main internal process to visualize
Slices Perspective	Direct use of slices from high-dimensional factor-product surfaces; no KDE or flow reconstruction inside the main forward/backward estimator.	Variable elimination, separator creation, f_{new} construction, conditional estimator, backward mixture of slices, MMD early stopping.
NF-iSAM	Bayes-tree clique conditional modeling using normalizing flows; full posterior recovered from learned clique conditionals and top-down ancestral sampling.	Changed subtree, leaf-to-root flow training, observation fixing, separator/conditional extraction, root-to-leaf sampling.
Smoothed Slices	Our proposed extension: reinterpret nested sampling paths as recursive conditional-expectation surfaces and reuse/cache them to reduce nested sampling burden.	Smoothing-surface recursion, finite-cardinality supports, active successor sets, cached surfaces, surface-guided defensive proposals.

4. MATLAB LaTeX interpreter policy

Hard UI rule
Every mathematical string shown in the app must be written in LaTeX form. Do not place mathematical formulas inside controls that cannot render LaTeX. Use <code>uicontrol</code> , <code>UIAxes title xlabel ylabel text</code> , <code>uihtml</code> , or image-rendered equation cards when needed. Buttons may use short plain text such as Run, Export, Next; mathematical explanations must be placed beside them in LaTeX-capable components.

Component type	Required LaTeX behavior	Implementation note
UIAxes	Titles, x labels, y labels, z labels, tick labels, legends, and text annotations use LaTeX interpreter.	Call <code>title(ax, '\$...\$', 'Interpreter', 'latex')</code> , <code>xlabel(...)</code> , <code>ylabel(...)</code> , <code>legend(..., 'Interpreter', 'latex')</code> , and set <code>ax.TickLabelInterpreter='latex'</code> .
uicontrol	Use <code>Interpreter = "latex"</code> for every label containing math or symbols.	Long equations should use <code>\$\$...\$\$</code> and be wrapped into equation cards.
uitable	Do not rely on LaTeX rendering inside table cells.	Use plain numeric diagnostics in tables; display equation meaning in a LaTeX label above/beside the table.
uibutton	Button captions stay short and plain unless current MATLAB release supports markup for the chosen button.	Do not put equations in buttons.
Diagnostic equations	Use a dedicated EquationPanel with <code>uicontrol/uihtml</code> or pre-rendered text.	Expose equations for P_{joint} , f_{new} , Eq. (16), R_r recursion, MMD, ESS, KL loss, and runtime/cardinality.

```
function applyLatexToAxes(ax)
    ax.TickLabelInterpreter = 'latex';
    ax.FontName = 'Times New Roman';
    ax.LineWidth = 1.0;
    grid(ax, 'on'); box(ax, 'on');
end

function setLatexTitle(ax, str)
    title(ax, str, 'Interpreter', 'latex');
end
```

```

function setLatexXY(ax, xstr, ystr)
    xlabel(ax, xstr, 'Interpreter','latex');
    ylabel(ax, ystr, 'Interpreter','latex');
end

function lbl = latexLabel(parent, txt, layout)
    lbl = uilabel(parent);
    lbl.Text = txt;
    lbl.Interpreter = 'latex';
    lbl.WordWrap = 'on';
    if nargin > 2, lbl.Layout = layout; end
end

```

5. Required app tabs and panels

Tab	Purpose	Required panels
Project Setup	Validate folders, load instruction PDFs, load literature list, choose output directory.	Project Health, Instruction Files, Literature Files, Results Folder, Random Seed, MATLAB Version.
Case Study	Define or load the SLAM experiment shared by all methods.	Dataset selector, map/obstacle view, measurement/noise model, data association setting, incremental timeline.
Compare Methods	Run Slices, NF-iSAM, and Smoothed Slices comparatively.	Method cards, Run All, Run Selected, reset seed, budget matching, runtime table.
Posterior Recovery	Show whether each method recovers the full posterior.	Samples, KDE/marginal curves, mode weights, uncertainty ellipses, MMD/RMSE traces.
Process Explorer	Drag sliders to see internal progress of inference.	Increment slider, method selector, stage slider, variable/cliue/surface panel, graph panel, equations panel.
Figure Generator	Export all visible and paper-style figures/diagnostics.	Checklist, Export All button, export format selector, diagnostics bundle preview.
Diagnostics	Analyze performance, cardinality, stability, and failure modes.	Runtime, memory, ESS, MMD, KL/loss, active set cardinalities, early stopping report.
Implementation Notes	Show algorithm equations and instructions from the project documents.	Paper-vs-method distinction, file provenance, method-specific implementation checklists.

6. Case studies required in the app

The app should be able to run progressively from small controlled examples to paper-style demonstrations. The same backend interface must be used for all case studies so the three methods can be compared without changing UI logic.

Case study	Purpose	Expected figure outputs
Local Eq. (16) demonstrator	Minimal $x_1 \rightarrow l_1 \rightarrow x_2$ graph. Used to show nested sampling versus conditional smoothing surface R1.	Elimination panels, R1 surface heatmap, nested sample tree, $f_{\text{new}}(x_2)$ comparison.
Four Doors / 1-D multimodal localization	Classical multimodal posterior recovery; should show mode preservation and mode collapse as new measurements arrive.	Marginal posterior curves, samples, MMD/RMSE vs sample count, mode weights over time.
Small range-only SLAM	2-D nonlinear range likelihoods with landmarks; can be run with known and ambiguous data association.	Map, robot trajectory, range circles/rays, landmark marginals, posterior samples, ambiguity lines.
Plaza2-style range-only dataset	Incremental real-world-style comparison with runtime and posterior evolution.	Landmark marginal evolution, RMSE/time per step, MMD if reference available,

		early stopping timeline.
Synthetic occupancy-like animation	Visual communication style similar to the user-provided MATLAB animation: rays, map panes, obstacles, robot path, goal/start.	Live map panels, rays, robot frame, occupancy map snapshots, exportable animation frames.

7. Backend architecture and file names

Use a thin App Designer frontend and a modular backend. The app file should not contain algorithm mathematics except dispatch and visualization callbacks. The backend must expose identical input/output contracts for all three methods.

```
src/
|-- SmoothedSlicesApp.mlapp          % optional App Designer frontend
|-- SmoothedSlicesAppProgrammatic.m  % preferred source-controlled frontend
|-- runSmoothedSlicesApp.m          % launch script
|
|-- +datasets/
|   |-- makeEq16DemoCase.m
|   |-- makeFourDoorsCase.m
|   |-- makeRangeOnlyCase.m
|   |-- loadPlaza2Case.m
|
|-- +methods/
|   |-- runSlicesMethod.m
|   |-- runNFISAMMethod.m
|   |-- runSmoothedSlicesMethod.m
|   |-- commonMethodConfig.m
|
|-- +methods/+slices/
|   |-- forwardElimination.m
|   |-- estimateNewFactor.m
|   |-- estimateConditional.m
|   |-- backwardMarginals.m
|   |-- mmdEarlyStop.m
|   |-- lemma1Sampler.m
|
|-- +methods/+nfisam/
|   |-- buildBayesTree.m
|   |-- generateTrainingSamples.m
|   |-- trainCliqueFlow.m
|   |-- extractConditionalSampler.m
|   |-- incrementalSubtreeUpdate.m
|   |-- samplePosteriorTopDown.m
|
|-- +methods/+smoothed/
|   |-- buildSamplingPath.m
|   |-- updateSmoothingSurfaces.m
|   |-- evaluateSurfaceRecursion.m
|   |-- fitSurfaceBasis.m
|   |-- surfaceGuidedProposal.m
|   |-- estimateNewFactorRCS.m
|
|-- +viz/
|   |-- plotFactorGraphStep.m
|   |-- plotBayesTreeStep.m
```

```

| |-- plotSmoothingSurfaceStep.m
| |-- plotPosteriorSamples.m
| |-- plotMarginalKDE.m
| |-- plotMetricsDashboard.m
| |-- exportAllFiguresAndDiagnostics.m
|
|-- +metrics/
| |-- computeRMSE.m
| |-- computeMMD.m
| |-- computeESS.m
| |-- computeModeWeights.m
| |-- computeRuntimeSummary.m
|
|-- +utils/
| |-- validateProjectTree.m
| |-- applyLatexToAxes.m
| |-- createRunFolder.m
| |-- saveRunDiagnostics.m
| |-- setRandomSeed.m

```

8. Unified method interface

All three methods must return the same result structure so that the comparative figures are generated by method-agnostic visualization code.

```

result = struct();
result.methodName = 'Slices' | 'NF-iSAM' | 'Smoothed Slices';
result.config = config;
result.case = caseData;
result.increment(k) = per-step records;
result.posterior.samples = samplesByStep;
result.posterior.marginals = marginalObjectsByStep;
result.metrics.rmse = rmseByStep;
result.metrics.mmd = mmdByStep;
result.metrics.runtime = runtimeByStep;
result.metrics.memory = memoryByStep;
result.metrics.cardinality = cardinalityDiagnostics;
result.process = processTrace; % for Process Explorer slider
result.figures = figureRegistry; % names/types to export
result.logs = text logs and warnings;

```

Field	Slices value	NF-iSAM value	Smoothed Slices value
processTrace.stageType	elimination/backward/ mmdStop	cliqueTraining/ flowExtraction/ topDownSampling	surfaceRecursion/ proposalGuidedSampling/ cacheUpdate
processTrace.nodeLabel	$\omega_j, S_j, F_{j-1}(\omega_j)$	clique C, F_C, S_C, z_C	x_i, R_r , active successor set $N_r(a)$
cardinality	$ X_j , S_j , samples $	$ training\ samples , cliques $, flow dimension	$ X_r , S , N_r , \Phi_s $
diagnostics	MMD early stopping, ESS, factor variance	training loss, KL objective, flow reuse, training time	surface error, variance removed, active-set cost, cache reuse

9. Comparative run pipeline

1. **Load project and validate folders.** Show which instruction documents and papers were found.
2. **Select case study.** Generate or load the same graph, measurements, noises, data-association conditions, and ground truth for all methods.
3. **Set budgets.** Choose samples/particles/support sizes/training samples/surface cardinalities. The UI must display all cardinalities in blue using |space letter| notation.
4. **Run Slices.** Use exact Slices implementation instructions, including forward pass, Lemma-1 sampling, conditional estimator, backward pass, and MMD early stopping.
5. **Run NF-iSAM.** Use exact NF-iSAM implementation instructions, including training sample simulation, normalizing-flow training, observation fixing, separator/conditional extraction, changed-subtree update, and top-down posterior sampling.
6. **Run Smoothed Slices.** Use the RCS implementation instructions, including surface recursion, active successor sets, optional basis/regression, cache reuse, and surface-guided defensive proposals.
7. **Compute diagnostics.** RMSE, MMD, mode weights, ESS, runtime, memory, flow loss, early stopping, cardinality, surface approximation error.
8. **Export figures.** Write PNG/PDF/SVG/MAT/CSV/JSON into a run-specific folder.

10. Process Explorer: required sliders and visual states

The Process Explorer is the key educational panel. It must let the user drag through internal computation, not only view final outputs.

Slider/control	Range	Effect on panels
Increment step k	1 to number of increments	Updates map, measurements available so far, posterior, metrics, and process traces.
Method selector	Slices / NF-iSAM / Smoothed Slices / Compare	Switches the process state representation. Compare mode synchronizes all methods at the same increment.
Process stage slider	Method-dependent stage count	Slices: elimination step j/backward step. NF-iSAM: clique/subtree/training/sampling step. Smoothed Slices: recursion level r/surface update.
Play/Pause	Timer-driven animation	Animates increments and process stages. Timer callback must not block UI; it should update one frame at a time.
Export current frame	Button	Saves the current map, process graph, equations panel, and metrics card.

11. Slices method process panels

- **Factor Graph Panel:** show G_{j-1} , highlight eliminated variable ω_j in red, removed factors F_{j-1} (ω_j) in orange, separator S_j in blue, and new factor f_{new} in green.
- **Equation Panel:** show $P_{\text{joint}}(\omega_j, S_j | D_j)$, chain-rule factorization, exact f_{new} integral, Monte Carlo slice estimator, conditional estimator, backward marginal estimator, and MMD early-stop equation.
- **Slice Samples Panel:** show sampled ω_j points and the resulting slices over S_j .
- **Backward Pass Panel:** show samples $S_j^{(n)}$ drawn from current marginal and the mixture of conditional slices.
- **Early Stop Panel:** show previous cached marginal, new marginal, MMD distance, threshold ϑ , number of MMD samples N_M , and stop/continue decision.

12. NF-iSAM method process panels

- **Bayes Tree Panel:** show cliques C , frontal variables F_C , separators S_C , child clique separator densities, and the changed subtree under new measurements.
- **Training Sample Generation Panel:** show selected loop-closing factors, observation variables O_C , intermediate density $p(O_C, S_C, F_C | z_C^*)$, and ancestral sample simulation.
- **Flow Training Panel:** show loss versus iteration, rolling relative loss change, number of training samples, RQ spline knots K , hidden units, and training stop condition.
- **Flow Extraction Panel:** show fixing $O_C = o_C$, then partitioning T_C into $T_{\{S_C\}}$ and $T_{\{F_C\}}$ to obtain $p(S_C | z_C)$ and $p(F_C | S_C, z_C)$.
- **Posterior Sampling Panel:** show root-to-leaf traversal and sampled frontal variables conditioned on separator realizations.
- **Incremental Reuse Panel:** show cliques outside the changed subtree as reused and cliques inside the subtree as retrained.

13. Smoothed Slices process panels

- **Sampling Path Panel:** show the Lemma-1 path $xi_0 \rightarrow xi_1 \rightarrow \dots \rightarrow xi_H$. Use red for the current level and blue for finite cardinality cards $|X_r|$, $|S|$, $|N_r|$.
- **Surface Recursion Panel:** show $R_r(xi_r, s)$ being computed from $R_{\{r+1\}}(xi_{\{r+1\}}, s)$ using active successors.
- **Surface Heatmap Panel:** for low-dimensional cases, show $R_1(x_1, x_2)$, $R_r(xi_r, s)$, or basis coefficients $\alpha_{\{r,k\}}(xi_r)$.
- **Nested-vs-Smoothed Panel:** left side shows branching nested particles; right side shows finite surface matrix R_r with sparse active successor sets.
- **Proposal Panel:** show $q_{\{r,paper\}}$, $q_{\{r,smooth\}}$, $q_{\{r,def\}}$, ESS, and whether more samples are needed.
- **Cache Panel:** show surfaces reused from $k-1$, surfaces updated at k , and memory contribution $|X_r| |S|$ or $|X_r| |Phi_s|$.

14. Figure Generator and diagnostics export subsystem

Required button

Create a clearly visible button named Export All Figures + Diagnostics. Pressing it must save all selected figure types, the current UI snapshot, app configuration, run metrics, posterior samples, method logs, and process traces.

```
function exportReport = exportAllFiguresAndDiagnostics(app)
    runDir = utils.createRunFolder(app.ProjectRoot, app.CurrentConfig);

    % 1. Export every registered UIAxes.
    for i = 1:numel(app.FigureRegistry.axes)
        ax = app.FigureRegistry.axes(i).handle;
        name = app.FigureRegistry.axes(i).name;
        exportgraphics(ax, fullfile(runDir, 'figures', name + ".png"), 'Resolution', 300);
        exportgraphics(ax, fullfile(runDir, 'figures', name + ".pdf"), 'ContentType', 'vector');
    end

    % 2. Export the whole app as a snapshot.
    exportapp(app.UIFigure, fullfile(runDir, 'app_snapshot.pdf'));
    exportapp(app.UIFigure, fullfile(runDir, 'app_snapshot.png'));

    % 3. Save numerical diagnostics.
    save(fullfile(runDir, 'run_state.mat'), 'app');
    writetable(app.MetricsTable, fullfile(runDir, 'metrics.csv'));
```

```

utils.writeJSON(fullfile(runDir,'config.json'), app.CurrentConfig);
utils.writeJSON(fullfile(runDir,'process_trace.json'), app.ProcessTrace);

% 4. Save textual summary.
exportReport = viz.writeDiagnosticsMarkdown(runDir, app.Results, app.CurrentConfig);
end

```

15. Figure types that must be generated

Figure family	Specific figures	Why it matters
Paper-structure figures	Factor graph -> Bayes net for Slices; Bayes tree with cliques for NF-iSAM; Lemma-1 sampling path; RCS surface recursion graph.	Explains algorithmic structure, not only numerical performance.
Posterior recovery figures	Posterior samples, KDE/marginal curves, marginal evolution over time, mode weights, uncertainty ellipses, donut/range likelihood shapes.	Shows whether the method recovers the distribution shape.
Classical SLAM map figures	Robot path, ground truth, odometry path, landmarks, range rays/circles, ambiguous association lines, occupancy-like map view.	Matches the visual style of the provided MATLAB animation and SLAM papers.
Performance figures	Runtime per increment, accumulated runtime, RMSE, joint MMD, marginal MMD, average MMD, memory, GPU/CPU split if available.	Compares online performance and posterior accuracy.
Method-specific diagnostics	Slices: ESS, MMD early stop, denominator stability. NF-iSAM: flow loss, flow dimension, samples, training time. Smoothed: $ X_r $, $ S $, $ N_r $, surface error, variance removed.	Distinguishes why one method succeeds or fails.
Animation frames	Every increment k and every selected process stage j/r/C. Export as PNG sequence and optional MP4.	Allows slide construction and visual storytelling.

16. Metrics and diagnostics definitions

Metric	Methods	Implementation requirement
RMSE	All methods	Compute against ground truth for robot poses and landmarks. Show empirical mean but warn that RMSE can hide posterior-shape errors.
MMD	All methods	Use for posterior shape comparison when reference samples exist. Allow kernel bandwidth selection and show it in diagnostics.
Mode weights	All methods	Cluster posterior samples or use known modes in Four Doors. Show mode collapse or mode loss.
Runtime	All methods	Separate total time, forward/upward time, backward/downward time, training time, sampling time, plotting time.
Memory	All methods	Estimate sample storage, factor cache, flow parameters, smoothing-surface matrices.
ESS / weight degeneracy	Slices and Smoothed	Effective sample size for importance or slice weights. Highlight low ESS in red.
Training loss	NF-iSAM	Loss curve and rolling relative change over latest windows. Stop when

		configured criterion is met.
Early stopping	Slices and Smoothed	MMD distance, threshold, sample count, stopped variable/subtree, saved backward steps.
Cardinality	All methods	Every finite space is displayed as space letter in blue: X _j , S _j , C , F _C , N _r , Phi _s .

17. Smoothed Slices implementation requirements

The Smoothed Slices backend must implement the RCS formulation as a proposed extension, not as part of the original paper. The UI must explicitly mark this distinction.

```
% Core RCS recursion
% Inputs:
% X{r}    finite support of xi_r, cardinality |X_r|
% S       separator support, cardinality |S|
% active{r,a} active successor set N_r(a), cardinality |N_r|
% P{r}(a,b) conditional smoothing weights/proposal weights
% A{r}(a,b,s) fusion tensor = Z_r(xi_r^a) a_r(xi_{r+1}^b, s)
% R{r+1}(b,s) deeper conditional smoothing surface
%
% Update:
% R{r}(a,s) = sum_{b in N_r(a)} P{r}(a,b) * A{r}(a,b,s) * R{r+1}(b,s)

function Rr = updateSurfaceSparse(Xr, Xnext, S, activeSet, P, A, Rnext)
    Br = numel(Xr); Rs = numel(S);
    Rr = zeros(Br, Rs);
    for a = 1:Br
        idx = activeSet{a};
        for rho = 1:Rs
            acc = 0;
            for jj = 1:numel(idx)
                b = idx(jj);
                acc = acc + P(a,b) * A(a,b,rho) * Rnext(b,rho);
            end
            Rr(a,rho) = acc;
        end
    end
end
```

- **Do not call it a message in the UI.** Use conditional smoothing surface, RCS surface, surface, or cached surface.
- **Always show what is exact and what is approximated.** Exact tower-property recursion is theoretical; finite R_r matrices are numerical approximations.
- **Cache surfaces.** At each increment k , save $R_r^{(k)}$ and reuse unchanged surfaces at $k+1$.
- **Use active successor sets.** The online gain must come from $|N_r| \ll |X_{r+1}|$, basis cardinality $|\Phi_s| \ll |S|$, or both.
- **Use fallback sampling.** If surface error is high, use paper-style samples or defensive proposal q_{def} .
- **Surface uncertainty.** Store a confidence/error estimate for every surface cell or basis coefficient when possible.

18. NF-iSAM implementation requirements

The NF-iSAM app mode must follow the paper-specified structure closely enough to be comparable, even if the first implementation uses simplified MATLAB flow objects or calls Python/PyTorch through MATLAB. The important point is to preserve the scheme.

- **Bayes tree.** Build or emulate cliques C with frontal variables F_C , separator S_C , and observations z_C .
- **Intermediate density.** Generate training samples from $p(O_C, S_C, F_C \mid z_C^*)$ by treating selected observations as unobserved variables to enable ancestral simulation.
- **Flow training.** Train a normalizing flow for the ordered vector (O_C, S_C, F_C) .
- **Observation fixing.** Fix $O_C = o_C$, then retrieve the desired flow over (S_C, F_C) .
- **Partition.** Partition the flow into separator density and conditional sampler $p(F_C \mid S_C, z_C)$.
- **Incremental update.** Retrain only cliques in the changed subtree; reuse flows outside the changed subtree.
- **Top-down sampling.** Sample from root to leaves using cached conditional samplers.

19. Slices implementation requirements

- **Forward pass.** For each eliminated variable ω_j , construct P_{joint} from the product of removed factors, define separator S_j , estimate $f_{\text{new}}(S_j \mid D_j)$, and store the conditional estimator.
- **Sampling route.** Use available unary factor if present; otherwise use the Lemma-1 structure induced by a previously eliminated neighboring variable.
- **Conditional estimator.** Use numerator factor product divided by the Monte Carlo denominator; track denominator stability.
- **Backward pass.** Compute marginals using samples of S_j from the current marginal and a mixture of conditional slices.
- **Early stopping.** Cache marginals; compute MMD between current and previous marginal; stop backward pass when MMD is below threshold. Show number of MMD samples N_M and threshold ϑ .

20. Animation and timer behavior

The app should feel like a MATLAB animation dashboard. Use a timer for playback, but never run expensive algorithms directly inside the timer callback. Precompute or execute one small process stage per timer tick.

```
function startAnimation(app)
    app.Timer = timer('ExecutionMode','fixedRate', ...
        'Period', app.AnimationPeriod, ...
        'TimerFcn', @(~,~)advanceOneFrame(app));
    start(app.Timer);
end

function advanceOneFrame(app)
    if app.IsBusy, return; end
    app.IsBusy = true;
    cleanup = onCleanup(@( )setfield(app,'IsBusy',false)); %#ok<SFLD>
    app.CurrentFrame = app.CurrentFrame + 1;
    updateProcessExplorer(app, app.CurrentFrame);
    drawnow limitrate;
end
```

21. Exact figure registry

Every UIAxes must be registered once. The Figure Generator exports only registered axes so that no figure is forgotten.

```

app.FigureRegistry.axes = [
    struct('name','map_current_increment','handle',app.AxMap),
    struct('name','posterior_samples_compare','handle',app.AxPosteriorSamples),
    struct('name','marginal_x_compare','handle',app.AxMarginalX),
    struct('name','marginal_y_compare','handle',app.AxMarginalY),
    struct('name','runtime_per_increment','handle',app.AxRuntime),
    struct('name','rmse_per_increment','handle',app.AxRMSE),
    struct('name','mmd_per_increment','handle',app.AxMMD),
    struct('name','slices_factor_graph_step','handle',app.AxSlicesGraph),
    struct('name','nfisam_bayes_tree_step','handle',app.AxNFISAMTree),
    struct('name','smoothed_surface_step','handle',app.AxSmoothedSurface),
    struct('name','cardinality_diagnostics','handle',app.AxCardinality),
    struct('name','early_stop_diagnostics','handle',app.AxEarlyStop)];

```

22. Acceptance checklist for the AI implementer

Requirement	Pass condition
Project hierarchy	Startup validates Instructions, Literature, presentation, data, results, src. Missing files are shown clearly.
LaTeX everywhere	All equations and math labels render through LaTeX-capable components. No raw unreadable equation strings in the UI.
Comparative run	One button runs all three methods on the same case and produces comparable posterior samples/marginals/metrics.
Process sliders	User can drag through incremental step and internal method stage.
Slices process	Variable elimination, separators, new factors, conditionals, backward pass, MMD early stop are visualized.
NF-iSAM process	Bayes tree clique training, observation fixing, flow extraction, changed-subtree reuse, and top-down sampling are visualized.
Smoothed Slices process	RCS surfaces, active sets, cache reuse, surface-guided proposals, and cardinality reductions are visualized.
Figure generator	Export All creates PNG/PDF of every registered axes plus app snapshot, MAT/CSV/JSON diagnostics, and a markdown summary.
Posterior recovery	At least one multimodal case shows posterior shape for all three methods and compares MMD/RMSE/runtime.
Reproducibility	Every run saves config, seed, MATLAB version, method budgets, and output paths.

23. AI implementation prompt

Prompt to hand to an implementation AI
Implement a MATLAB App Designer or programmatic MATLAB app named Smoothed Slices. The app must compare Slices Perspective, NF-iSAM, and Smoothed Slices on shared non-Gaussian SLAM case studies. It must validate the project folder hierarchy, use LaTeX interpretation for all mathematical UI content, include a Process Explorer with sliders for incremental inference and method-specific internal stages, and include an Export All Figures + Diagnostics button that saves every figure and numerical diagnostic to a run folder. Keep algorithm code modular under +methods, plotting under +viz, metrics under +metrics, datasets under +datasets, and utilities under +utils. The app must show posterior recovery, not only point estimate quality.

24. References and implementation sources

- Shienman, Levy-Or, Kaess, and Indelman, "A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces," IROS 2024. Use for exact Slices forward/backward estimator, Lemma 1 sampleability logic, Eq. (16) example, and MMD early stopping.
- Huang et al., "Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows," IEEE Transactions on Robotics, 2023. Use for Bayes-tree clique conditional modeling, Algorithms 1-3, normalizing-flow extraction, changed-subtree incremental update, and implementation settings.
- Recursive Conditional-Smoothing Slices formulation note. Use for the proposed Smoothed Slices method: tower-property interpretation, conditional smoothing surfaces, finite-cardinality implementation, active successor sets, and surface-guided proposals.
- MathWorks documentation: UIAxes/uiaxes, uilabel Interpreter, uigridlayout/GridLayout, timer in App Designer, exportgraphics, exportapp. Use official documentation to implement MATLAB UI behavior and figure export.